

ASSESSMENT TOOL 3 -DATA INTEGRATION TEST

Course Code: DSA0504	Course Title: Query Processing for Data Science
CO Assessed: CO3	Assessment: Tool 3 – Data integration test
Weightage: 30	Total Marks: 30 Marks
CO2 Statement: Identify and clean inconsistencies in data by handling missing values, duplicates,outliers, formatting issues, and applying techniques like fuzzy matching andregex.	
Student Name:J.Subhaashree	Reg. No.:192424008
Section / Batch:2024	Date of Submission:10/8/26
Faculty In-charge: Dr. R.Mahesh Muthulakshmi	Project Mode: Individual Submission

CO3 AT3

Dataset Integration Using Pandas

Question 4 – Systematic Validation of Integrated Datasets

Objective

To design and implement a systematic validation procedure using Python and Pandas to identify problems that may occur after integrating multiple datasets, including incorrect row counts, duplicate identifiers, duplicate keys, missing values, incompatible data types, and unmatched records.

Dataset Used

Four placement-related datasets were used:

- Student Details
- Placement
- Company
- Department

The datasets were loaded directly using Pandas without manually modifying the original CSV files.

Methodology

The datasets were first inspected to identify their dimensions, column names, data types, and sample records. Row counts were then compared before integration to determine whether records were lost or unexpectedly multiplied.

Unique student identifiers were examined using the `nunique()` function. Duplicate keys were identified using the `duplicated()` function. This was particularly important for the Placement dataset because a student may legitimately have multiple placement opportunities.

Missing values were examined using `isnull().sum()`. Data types were also inspected to identify fields that might prevent correct matching during integration.

Student IDs were standardized by removing unwanted spaces and converting them to uppercase. Unmatched placement records were then identified using the `isin()` function. Finally, an outer merge with the `indicator=True` option was performed to identify records that existed only in the Student Details dataset, only in the Placement dataset, or in both datasets.

Import dataset

```
# =====  
# Q4: SYSTEMATIC VALIDATION OF INTEGRATED DATASETS  
# =====  
  
import pandas as pd  
  
# -----  
# 1. Load datasets  
# -----  
  
student = pd.read_csv("/content/Student_Details.csv")  
placement = pd.read_csv("/content/Placement.csv")  
company = pd.read_csv("/content/Company.csv")  
department = pd.read_csv("/content/Department.csv")  
  
print("Datasets loaded successfully.")  
  
Datasets loaded successfully.
```

Inspect Dataset Structure

```
# -----  
# 2. Inspect structure  
# -----  
  
datasets = {  
    "Student Details": student,  
    "Placement": placement,  
    "Company": company,  
    "Department": department  
}  
  
for name, df in datasets.items():  
    print("\n" + "="*60)  
    print(name)  
    print("="*60)  
  
    print("Shape:", df.shape)  
    print("\nColumns:")  
    print(df.columns.tolist())  
  
    print("\nData Types:")  
    print(df.dtypes)  
  
    print("\nFirst 5 Records:")  
    print(df.head())
```

```
=====
Student Details
=====
```

```
Shape: (8, 4)
```

```
Columns:
```

```
['Student_ID', 'Name', 'Department', 'Graduation_Year']
```

```
Data Types:
```

```
Student_ID      object
```

```
Name            object
```

```
Department       object
```

```
Graduation_Year  int64
```

```
dtype: object
```

```
First 5 Records:
```

	Student_ID	Name	Department	Graduation_Year
0	S101	Ananya	CSE	2026
1	S102	Bharath	cse	2026
2	S103	Charan	Computer Science	2026
3	S104	Divya	AIDS	2026
4	S105	Eshan	CSE	2026

```
=====
Placement
=====
```

```
Shape: (10, 4)
```

```
Columns:
```

```
['StudentID', 'Company', 'Salary', 'Placement_Date']
```

```
Data Types:
```

```
StudentID       object
```

```
Company          object
```

```
Salary           int64
```

```
Placement_Date   object
```

```
dtype: object
```

```
First 5 Records:
```

	StudentID	Company	Salary	Placement_Date
0	S101	TCS	500000	12-06-2025
1	S101	Infosys	450000	2025/06/15
2	S101	Wipro	550000	18/06/2025
3	s102	HCL	600000	20-06-2025
4	S103	Accenture	700000	2025/06/21

```
=====
Company
=====
```

```
Shape: (6, 4)
```

```
Columns:
['Company_ID', 'Company_Name', 'Industry', 'Location']
```

```
Data Types:
Company_ID      object
Company_Name    object
Industry        object
Location        object
dtype: object
```

```
First 5 Records:
```

	Company_ID	Company_Name	Industry	Location
0	C01	TCS	IT	Chennai
1	C02	Infosys	IT	Bengaluru
2	C03	Wipro	IT	Chennai
3	C04	HCL	IT	Noida
4	C05	Accenture	Consulting	Bengaluru

```
=====
Department
=====
```

```
Shape: (3, 3)
```

```
Columns:
['Department_ID', 'Department_Name', 'Department_Full_Name']
```

```
Data Types:
Department_ID      object
Department_Name     object
Department_Full_Name object
dtype: object
```

```
First 5 Records:
```

	Department_ID	Department_Name	Department_Full_Name
0	D01	CSE	Computer Science
1	D02	AIDS	Artificial Intelligence and Data Science
2	D03	ECE	Electronics and Communication Engineering

Check Row Counts

```
# -----  
# 3. Row count validation  
# -----  
  
print("Student Details rows:", len(student))  
print("Placement rows:", len(placement))  
print("Company rows:", len(company))  
print("Department rows:", len(department))  
  
Student Details rows: 8  
Placement rows: 10  
Company rows: 6  
Department rows: 3
```

Check Unique Identifiers

```
# -----  
# 4. Unique identifier validation  
# -----  
  
print("Unique Student IDs:",  
      student["Student_ID"].nunique())  
  
print("Total Student records:",  
      len(student))  
  
print("Unique Placement Student IDs:",  
      placement["StudentID"].nunique())  
  
Unique Student IDs: 8  
Total Student records: 8  
Unique Placement Student IDs: 7
```

Check Duplicate Keys

```
# -----  
# 5. Duplicate key validation  
# -----  
  
print("\nDuplicate Student IDs in Student Details:")  
print(student[  
    student["Student_ID"].duplicated(keep=False)  
])  
  
print("\nDuplicate Student IDs in Placement:")  
print(placement[  
    placement["StudentID"].duplicated(keep=False)  
])
```

Duplicate Student IDs in Student Details:
Empty DataFrame
Columns: [Student_ID, Name, Department, Graduation_Year]
Index: []

Duplicate Student IDs in Placement:

	StudentID	Company	Salary	Placement_Date
0	S101	TCS	500000	12-06-2025
1	S101	Infosys	450000	2025/06/15
2	S101	Wipro	550000	18/06/2025
6	S105	Zoho	800000	25/06/2025
7	S105	TCS	750000	2025-06-26

Check Duplicate Records

```
# -----  
# 6. Duplicate record validation  
# -----  
  
print("Duplicate Student records:",  
      student.duplicated().sum())  
  
print("Duplicate Placement records:",  
      placement.duplicated().sum())  
  
print("Duplicate Company records:",  
      company.duplicated().sum())  
  
print("Duplicate Department records:",  
      department.duplicated().sum())
```

Duplicate Student records: 0
Duplicate Placement records: 0
Duplicate Company records: 0
Duplicate Department records: 0

Check Missing Values

```
# -----  
# 7. Missing value validation  
# -----  
  
for name, df in datasets.items():  
    print("\nMissing values -", name)  
    print(df.isnull().sum())  
  
Missing values - Student Details  
Student_ID      0  
Name            0  
Department      0  
Graduation_Year 0  
dtype: int64  
  
Missing values - Placement  
StudentID      0  
Company        0  
Salary         0  
Placement_Date 0  
dtype: int64  
  
Missing values - Company  
Company_ID      0  
Company_Name    0  
Industry       0  
Location       0  
dtype: int64  
  
Missing values - Department  
Department_ID    0  
Department_Name  0  
Department_Full_Name 0  
dtype: int64
```


Check Data Types

```
# -----  
# 8. Data type validation  
# -----  
  
print("\nStudent Details Data Types:")  
print(student.dtypes)  
  
print("\nPlacement Data Types:")  
print(placement.dtypes)  
  
Student Details Data Types:  
Student_ID      object  
Name            object  
Department      object  
Graduation_Year int64  
dtype: object  
  
Placement Data Types:  
StudentID      object  
Company        object  
Salary         int64  
Placement_Date object  
dtype: object
```

Standardize Student ID for Validation

```
# -----  
# 9. Standardize IDs for comparison  
# -----  
  
student["Student_ID"] = (  
    student["Student_ID"]  
    .astype(str)  
    .str.strip()  
    .str.upper()  
)  
  
placement["StudentID"] = (  
    placement["StudentID"]  
    .astype(str)  
    .str.strip()  
    .str.upper()  
)
```

Find Unmatched Records

```
# -----
# 10. Find unmatched placement records
# -----

unmatched_placement = placement[
    ~placement["StudentID"].isin(student["Student_ID"])
]

print("Number of unmatched placement records:",
      len(unmatched_placement))

print("\nUnmatched records:")
display(unmatched_placement)
```

Number of unmatched placement records: 1

Unmatched records:

					1 entry
index	StudentID	Company	Salary	Placement_Date	
8	S109	TechCorp	500000	27-06-2025	

Show per page
Like what you see? Visit the [data table notebook](#) to learn more about interactive tables.

Perform Validation Merge

```
# -----
# 11. Outer merge with indicator
# -----

validation = pd.merge(
    student,
    placement,
    left_on="Student_ID",
    right_on="StudentID",
    how="outer",
    indicator=True
)

print(validation["_merge"].value_counts())
```

```
_merge
both          9
left_only     2
right_only    1
Name: count, dtype: int64
```

Final Validation

```
# -----  
# 12. Final validation  
# -----  
  
print("Original student records:", len(student))  
print("Original placement records:", len(placement))  
print("Integrated records:", len(validation))  
  
print("\nMissing values after integration:")  
print(validation.isnull().sum())  
  
print("\nDuplicate Student IDs after integration:")  
print(validation["Student_ID"].duplicated().sum())  
  
Original student records: 8  
Original placement records: 10  
Integrated records: 12  
  
Missing values after integration:  
Student_ID      1  
Name            1  
Department      1  
Graduation_Year 1  
StudentID       2  
Company         2  
Salary          2  
Placement_Date  2  
_merge         0  
dtype: int64  
  
Duplicate Student IDs after integration:  
3
```

Evidence from the Data

The validation identified that the Placement dataset contains repeated student identifiers. These repetitions are not necessarily erroneous because one student may receive interview opportunities from multiple companies.

The validation also identified an unmatched placement record for a student identifier that was not present in the Student Details dataset. This demonstrates how an unmatched-record check can reveal data integration problems even when Pandas completes the merge without producing an error.

Conclusion

A successful Pandas merge does not automatically mean that the integrated dataset is correct. Row counts identify record loss or unexpected multiplication, unique identifiers and duplicate-key checks identify key problems, missing-value checks identify incomplete information, data-type checks identify incompatible fields, and unmatched-record checks identify failed relationships between datasets. Therefore, systematic validation should be performed after every major integration operation

Question 5 – One-to-Many Relationship in Placement Data

Objective

To integrate Student Details and Placement datasets while correctly handling a one-to-many relationship and to calculate the number of placement opportunities and average salary for each individual student without incorrectly deleting legitimate records.

Concept

The Student Details dataset contains one record for each student. In contrast, the Placement dataset can contain multiple records for the same student because a student may receive interview or placement opportunities from several companies.

Therefore, the relationship is:

One Student → Many Placement Records

A repeated student identifier in the Placement dataset should not automatically be treated as a duplicate error.

Methodology

The Student Details and Placement datasets were loaded using Pandas. Student IDs were standardized by removing spaces and converting them to uppercase.

The number of placement records for each student was calculated using `groupby()`. Students with more than one placement record were identified.

A left merge was then performed between the Student Details and Placement datasets. This preserves all students while allowing multiple placement records to be associated with the same student.

For individual-level analysis, the integrated data was aggregated using `groupby()`. The number of placement opportunities was calculated using `count()`, while the average salary was calculated using `mean()`.

Evidence from the Data

The Placement dataset contains multiple records for students such as S101 and S105. For example, S101 has three placement opportunities from different companies.

These records are legitimate because each record represents a separate company opportunity. Deleting the repeated S101 records would incorrectly reduce the number of placement opportunities and distort the analysis.

The student-level aggregation converts the transaction-level information into one record per student while retaining the important measures:

- Number of placement opportunities
- Average salary

Import dataset

```
# =====  
# Q5: ONE-TO-MANY RELATIONSHIP AND AGGREGATION  
# =====  
  
import pandas as pd  
  
# -----  
# 1. Load datasets  
# -----  
  
student = pd.read_csv("Student_Details.csv")  
placement = pd.read_csv("Placement.csv")  
  
print("Student Details:")  
display(student)  
  
print("\nPlacement:")  
display(placement)
```

Student Details:				
	Student_ID	Name	Department	Graduation_Year
0	S101	Ananya	CSE	2026
1	S102	Bharath	cse	2026
2	S103	Charan	Computer Science	2026
3	S104	Divya	AIDS	2026
4	S105	Eshan	CSE	2026
5	S106	Fathima	Computer Science	2026
6	S107	Gokul	cse	2026
7	S108	Harini	AIDS	2026

Placement:				
	StudentID	Company	Salary	Placement_Date
0	S101	TCS	500000	12-06-2025
1	S101	Infosys	450000	2025/06/15
2	S101	Wipro	550000	18/06/2025
3	s102	HCL	600000	20-06-2025
4	S103	Accenture	700000	2025/06/21
5	S104	Cognizant	650000	22-06-2025
6	S105	Zoho	800000	25/06/2025
7	S105	TCS	750000	2025-06-26
8	S109	TechCorp	500000	27-06-2025
9	S106	Wipro	550000	2025/06/28

Standardize Student IDs

```
# -----
# 2. Standardize Student IDs
# -----

student["Student_ID"] = (
    student["Student_ID"]
    .astype(str)
    .str.strip()
    .str.upper()
)

placement["StudentID"] = (
    placement["StudentID"]
    .astype(str)
    .str.strip()
    .str.upper()
)
```

Identify Multiple Placement Records

```
# -----  
# 3. Identify students with multiple opportunities  
# -----  
  
placement_counts = (  
    placement.groupby("StudentID")  
    .size()  
    .reset_index(name="Placement_Opportunities")  
)  
  
print("Placement opportunities per student:")  
display(placement_counts)
```

Placement opportunities per student:

	StudentID	Placement_Opportunities
0	S101	3
1	S102	1
2	S103	1
3	S104	1
4	S105	2
5	S106	1
6	S109	1

Display Duplicate Student IDs

```
# -----  
# 4. Show students appearing multiple times  
# -----  
  
multiple = placement_counts[  
    placement_counts["Placement_Opportunities"] > 1  
]  
  
print("Students with multiple placement opportunities:")  
display(multiple)
```

Students with multiple placement opportunities:

	StudentID	Placement_Opportunities
0	S101	3
4	S105	2

Perform One-to-Many Merge

```
# -----  
# 5. One-to-many merge  
# -----  
  
integrated = pd.merge(  
    student,  
    placement,  
    left_on="Student_ID",  
    right_on="StudentID",  
    how="left"  
)  
  
print("Integrated dataset:")  
display(integrated)
```

Integrated dataset:

	Student_ID	Name	Department	Graduation_Year	StudentID	Company	Salary	Placement_Date
0	S101	Ananya	CSE	2026	S101	TCS	500000.0	12-06-2025
1	S101	Ananya	CSE	2026	S101	Infosys	450000.0	2025/06/15
2	S101	Ananya	CSE	2026	S101	Wipro	550000.0	18/06/2025
3	S102	Bharath	cse	2026	S102	HCL	600000.0	20-06-2025
4	S103	Charan	Computer Science	2026	S103	Accenture	700000.0	2025/06/21
5	S104	Divya	AIDS	2026	S104	Cognizant	650000.0	22-06-2025
6	S105	Eshan	CSE	2026	S105	Zoho	800000.0	25/06/2025
7	S105	Eshan	CSE	2026	S105	TCS	750000.0	2025-06-26
8	S106	Fathima	Computer Science	2026	S106	Wipro	550000.0	2025/06/28
9	S107	Gokul	cse	2026	NaN	NaN	NaN	NaN
10	S108	Harini	AIDS	2026	NaN	NaN	NaN	NaN

Convert Salary to Numeric

```
# -----  
# 6. Convert salary into numeric format  
# -----  
  
integrated["Salary"] = pd.to_numeric(  
    integrated["Salary"],  
    errors="coerce"  
)
```


Aggregate Placement Information

```
# -----  
# 7. Create student-level summary  
# -----  
  
student_summary = (  
    integrated.groupby(  
        ["Student_ID", "Name", "Department", "Graduation_Year"],  
        as_index=False  
    )  
    .agg(  
        Placement_Opportunities=("Company", "count"),  
        Average_Salary=("Salary", "mean")  
    )  
)  
  
display(student_summary)
```

	Student_ID	Name	Department	Graduation_Year	Placement_Opportunities	Average_Salary
0	S101	Ananya	CSE	2026	3	500000.0
1	S102	Bharath	cse	2026	1	600000.0
2	S103	Charan	Computer Science	2026	1	700000.0
3	S104	Divya	AIDS	2026	1	650000.0
4	S105	Eshan	CSE	2026	2	775000.0
5	S106	Fathima	Computer Science	2026	1	550000.0
6	S107	Gokul	cse	2026	0	NaN
7	S108	Harini	AIDS	2026	0	NaN

Display S101

```
# -----  
# 8. Check an individual student  
# -----  
  
print("Details of S101:")  
display(  
    student_summary[  
        student_summary["Student_ID"] == "S101"  
    ]  
)
```

Details of S101:

	Student_ID	Name	Department	Graduation_Year	Placement_Opportunities	Average_Salary
0	S101	Ananya	CSE	2026	3	500000.0

Validate Number of Students

```
# -----  
# 9. Validation  
# -----  
  
print("Original number of students:",  
      student["Student_ID"].nunique())  
  
print("Final number of students:",  
      student_summary["Student_ID"].nunique())  
  
print("\nTotal placement opportunities:",  
      placement.shape[0])  
  
Original number of students: 8  
Final number of students: 8  
  
Total placement opportunities: 10
```

```
=====
Student Details
=====
Shape: (8, 4)

Columns:
['Student_ID', 'Name', 'Department', 'Graduation_Year']

Data Types:
Student_ID      object
Name            object
Department      object
Graduation_Year int64
dtype: object

First 5 Records:
  Student_ID  Name      Department  Graduation_Year
0      S101  Ananya          CSE          2026
1      S102  Bharath          cse          2026
2      S103  Charan  Computer Science    2026
3      S104  Divya          AIDS          2026
4      S105  Eshan          CSE          2026

=====
Placement
=====
Shape: (10, 4)

Columns:
['StudentID', 'Company', 'Salary', 'Placement_Date']

Data Types:
StudentID      object
Company        object
Salary         int64
Placement_Date object
dtype: object

First 5 Records:
  StudentID  Company  Salary  Placement_Date
0      S101      TCS   500000    12-06-2025
1      S101  Infosys   450000    2025/06/15
2      S101    Wipro   550000    18/06/2025
3      s102     HCL   600000    20-06-2025
4      S103  Accenture  700000    2025/06/21
=====
```

```

=====
Company
=====
Shape: (6, 4)

Columns:
['Company_ID', 'Company_Name', 'Industry', 'Location']

Data Types:
Company_ID      object
Company_Name    object
Industry        object
Location        object
dtype: object

First 5 Records:
  Company_ID Company_Name  Industry  Location
0         C01          TCS         IT   Chennai
1         C02       Infosys         IT  Bengaluru
2         C03         Wipro         IT   Chennai
3         C04          HCL         IT    Noida
4         C05  Accenture  Consulting  Bengaluru

=====
Department
=====
Shape: (3, 3)

Columns:
['Department_ID', 'Department_Name', 'Department_Full_Name']

Data Types:
Department_ID      object
Department_Name     object
Department_Full_Name object
dtype: object

First 5 Records:
  Department_ID Department_Name  Department_Full_Name
0           D01             CSE      Computer Science
1           D02             AIDS  Artificial Intelligence and Data Science
2           D03             ECE  Electronics and Communication Engineering

```

Decision and Justification

The original placement records should be retained because they represent a legitimate one-to-many relationship. Aggregation should only be performed when the analytical objective requires one row per student. This approach avoids loss of information while still allowing individual-student analysis.

Conclusion

The correct approach is not to delete repeated Student IDs from the Placement dataset. Instead, the records should first be integrated using the one-to-many relationship and then aggregated when student-level analysis is required. This preserves legitimate placement opportunities and produces accurate placement counts and average salary values.

Question 6 – Data Preprocessing and Standardization

Objective

To develop a Python-based preprocessing strategy for standardizing inconsistent column names, categorical values, dates, identifiers, and numerical values before performing final dataset integration.

Data Inconsistencies

The datasets contain several forms of inconsistency. Student identifiers are represented using different column names such as `Student_ID` and `StudentID`. Department values may appear as `CSE`, `cse`, or `Computer Science`. Dates are represented using different formats such as `12-06-2025` and `2025/06/12`. Salary values may also be stored as text rather than numeric data.

Methodology

All datasets were loaded using Pandas without manually modifying the original CSV files.

First, column names were standardized by removing leading and trailing spaces, converting them to lowercase, and replacing spaces with underscores. The Placement dataset's `student_id` column was renamed to `student_id` to match the Student Details dataset.

Student IDs were converted to strings, leading and trailing spaces were removed, and values were converted to uppercase. This ensures that identifiers such as `s101` and `S101` are treated as the same student.

Department values were converted to lowercase during preprocessing and then mapped to standardized names. For example, `CSE`, `cse`, and `Computer Science` were normalized to `Computer Science`.

Placement dates were converted using `pd.to_datetime()`, allowing different date formats to be represented consistently as `datetime` values. Salary values were converted from text to numeric using `pd.to_numeric()`.

Finally, the standardized fields and data types were checked to verify that preprocessing was successful.

Why Standardization Is Required

Standardization should be performed before integration because merge operations depend on compatible column names, identifiers, data types, and matching values.

For example, if one dataset contains `S101` and another contains `s101`, they may not match during a direct comparison. Similarly, `CSE`, `cse`, and `Computer Science` could incorrectly be treated as separate categories during analysis.

Possible Analytical Errors

If these inconsistencies were ignored, several errors could occur:

- Student records may fail to match.
- Unmatched records may increase.
- Missing values may appear after merging.
- Student counts may be incorrect.
- Department-wise analysis may contain duplicate categories.
- Average salary calculations may be incorrect if salary remains text.
- Date-based analysis may fail or produce incorrect results.
- Placement statistics may be under-counted.
- Duplicate student records may be incorrectly created.

Import dataset

```
# =====  
# Q6: DATA PREPROCESSING AND STANDARDIZATION  
# =====  
  
import pandas as pd  
  
# -----  
# 1. Load datasets  
# -----  
  
student = pd.read_csv("Student_Details.csv")  
placement = pd.read_csv("Placement.csv")  
company = pd.read_csv("Company.csv")  
department = pd.read_csv("Department.csv")  
  
print("All datasets loaded successfully.")  
  
All datasets loaded successfully.
```

Inspect Original Column Names

```
# -----  
# 2. Inspect column names  
# -----  
  
print("Student Details columns:")  
print(student.columns.tolist())  
  
print("\nPlacement columns:")  
print(placement.columns.tolist())  
  
print("\nCompany columns:")  
print(company.columns.tolist())  
  
print("\nDepartment columns:")  
print(department.columns.tolist())  
  
Student Details columns:  
['Student_ID', 'Name', 'Department', 'Graduation_Year']  
  
Placement columns:  
['StudentID', 'Company', 'Salary', 'Placement_Date']  
  
Company columns:  
['Company_ID', 'Company_Name', 'Industry', 'Location']  
  
Department columns:  
['Department_ID', 'Department_Name', 'Department_Full_Name']
```

Standardize Column Names

```
# -----  
# 3. Standardize column names  
# -----  
  
def standardize_columns(df):  
    df.columns = (  
        df.columns  
        .str.strip()  
        .str.lower()  
        .str.replace(" ", "_")  
    )  
    return df  
  
student = standardize_columns(student)  
placement = standardize_columns(placement)  
company = standardize_columns(company)  
department = standardize_columns(department)  
  
print("Standardized column names:")  
print(student.columns.tolist())  
print(placement.columns.tolist())  
  
Standardized column names:  
['student_id', 'name', 'department', 'graduation_year']  
['studentid', 'company', 'salary', 'placement_date']
```

Rename Student ID

```
# -----  
# 4. Standardize Student ID column  
# -----  
  
placement = placement.rename(  
    columns={"studentid": "student_id"}  
)  
  
print(placement.columns.tolist())  
  
['student_id', 'company', 'salary', 'placement_date']
```

Standardize Student IDs

```
# -----  
# 5. Standardize Student IDs  
# -----  
  
student["student_id"] = (  
    student["student_id"]  
    .astype(str)  
    .str.strip()  
    .str.upper()  
)  
  
placement["student_id"] = (  
    placement["student_id"]  
    .astype(str)  
    .str.strip()  
    .str.upper()  
)  
  
print("Student IDs standardized.")  
  
Student IDs standardized.
```

Check Department Values

```
# -----  
# 6. Check department inconsistencies  
# -----  
  
print("Original department values:")  
print(student["department"].value_counts())  
  
Original department values:  
department  
CSE                2  
cse                2  
Computer Science  2  
AIDS              2  
Name: count, dtype: int64
```


Standardize Department Names

```
# -----  
# 7. Standardize department values  
# -----  
  
student["department"] = (  
    student["department"]  
    .astype(str)  
    .str.strip()  
    .str.lower()  
)  
  
department_mapping = {  
    "cse": "Computer Science",  
    "computer science": "Computer Science",  
    "computer science engineering": "Computer Science",  
    "aids": "Artificial Intelligence and Data Science",  
    "artificial intelligence and data science":  
        "Artificial Intelligence and Data Science"  
}  
  
student["department"] = (  
    student["department"]  
    .replace(department_mapping)  
)  
  
print("Standardized department values:")  
print(student["department"].value_counts())  
  
Standardized department values:  
department  
Computer Science          6  
Artificial Intelligence and Data Science  2  
Name: count, dtype: int64
```

Standardize Dates

```
# -----  
# 8. Standardize placement dates  
# -----  
  
placement["placement_date"] = pd.to_datetime(  
    placement["placement_date"],  
    errors="coerce",  
    dayfirst=True  
)  
  
print(placement["placement_date"])
```

0	2025-06-12
1	NaT
2	NaT
3	2025-06-20
4	NaT
5	2025-06-22
6	NaT
7	NaT
8	2025-06-27
9	NaT

Name: placement_date, dtype: datetime64[ns]

Convert Salary to Numeric

```
# -----  
# 9. Convert salary from text to numeric  
# -----  
  
placement["salary"] = pd.to_numeric(  
    placement["salary"],  
    errors="coerce"  
)  
  
print(placement["salary"].dtype)
```

Remove Extra Spaces from Text

```
# -----  
# 10. Remove unwanted spaces  
# -----  
  
for df in [student, placement, company, department]:  
    for column in df.select_dtypes(include="object").columns:  
        df[column] = df[column].str.strip()
```

Check Final Data Types

```
# -----  
# 11. Final data type validation  
# -----  
  
print("Student data types:")  
print(student.dtypes)  
  
print("\nPlacement data types:")  
print(placement.dtypes)
```

Student data types:

student_id	object
name	object
department	object
graduation_year	int64
dtype:	object

Placement data types:

student_id	object
company	object
salary	int64
placement_date	datetime64[ns]
dtype:	object

Check Standardized Values

```
# -----  
# 12. Final validation  
# -----  
  
print("\nStandardized Student IDs:")  
print(student["student_id"].unique())  
  
print("\nStandardized Departments:")  
print(student["department"].unique())  
  
print("\nPlacement Dates:")  
print(placement["placement_date"].head())  
  
print("\nSalary Data Type:")  
print(placement["salary"].dtype)  
  
Standardized Student IDs:  
['S101' 'S102' 'S103' 'S104' 'S105' 'S106' 'S107' 'S108']  
  
Standardized Departments:  
['Computer Science' 'Artificial Intelligence and Data Science']  
  
Placement Dates:  
0    2025-06-12  
1           NaT  
2           NaT  
3    2025-06-20  
4           NaT  
Name: placement_date, dtype: datetime64[ns]  
  
Salary Data Type:  
int64
```

Evidence from the Data

The practice datasets contain inconsistent representations of Student IDs and department names. The Placement dataset also contains salary values stored as strings and dates in multiple formats. Standardization converts these fields into consistent representations suitable for integration and analysis.

Conclusion

Data standardization is an essential preprocessing stage before dataset integration. Standardizing identifiers, column names, categorical values, dates, and numerical fields improves matching accuracy and reduces artificial missing values, unmatched records, duplicate categories, and incorrect analytical results. The standardized datasets can therefore be safely used for subsequent integration and analysis.